PROJECT REPORT OF INDUSTRY ORIENTED HANDS ON EXPERIENCE (IOHE)

ON

Secret Box – Anonymous Messaging Platform

submitted in partial fulfilment of the requirements for the award of degree of

BACHELOR OF ENGINEERING

In

COMPUTER SCIENCE AND ENGINEERING

Submitted by:
Pratik Raj
2211981272

Supervised By:
Dr. Preeti Saini
Assistant Professor
Chitkara University, Punjab

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

CHITKARA UNIVERSITY INSTITUTE OF ENGINEERING AND TECHNOLOGY

CHITKARA UNIVERSITY, PUNJAB, INDIA

CONTENTS

DECLARATION

I hereby certify that the work which is being presented in the project report entitled **"Secret Box – Anonymous Messaging Platform"** in partial fulfilment of requirement for the award of the degree of Bachelor of Engineering (Computer Science and Engineering) submitted in the department of Computer Science and Engineering at Chitkara University Institute of Engineering and Technology, Chitkara University, Punjab, India, is an authentic record of my own work carried out under the supervision of Dr. Preeti Saini. The matter presented in this project report has not been submitted in any other university/institute for the award of any degree.

Place: Rajpura **Pratik Raj**

Date: …… 2211981272

This is to certify that the above statement made by the candidate is correct to the best of my knowledge and belief.

Dr. Preeti Saini
Assistant Professor
Department of Computer Science and Engineering
Chitkara University Institute of Engineering and Technology,
Chitkara University, Punjab, India

ACKNOWLEDGEMENT

I would like to express my sincere gratitude to all those who have contributed to the successful completion of this project, "Secret Box – Anonymous Messaging Platform."

First and foremost, I extend my heartfelt thanks to my mentor for their invaluable guidance, constant encouragement, and constructive feedback throughout the development of this project. Their expertise and mentorship played a pivotal role in shaping the direction and quality of this work.

I am also grateful to the Department of Computer Science and Engineering at Chitkara University Institute of Engineering and Technology for providing the academic environment, infrastructure, and resources necessary for this project.

I would like to thank the open-source community for the excellent tools, libraries, and frameworks—including Next.js, NextAuth.js, MongoDB, and Tailwind CSS—that made this project possible.

Finally, I express my appreciation to my family and peers for their unwavering support and motivation during this journey.

ABSTRACT

Secret Box is a privacy-first, full-stack web application designed to facilitate anonymous communication through a secure and user-friendly platform. The application enables users to receive anonymous messages via unique, shareable links without requiring senders to create an account or log in, thereby ensuring complete sender anonymity.

Built using the Next.js framework with TypeScript, the platform leverages a modern technology stack including NextAuth.js for multi-provider authentication (credentials, Google, and GitHub), MongoDB with Mongoose for database management, Zod for schema-based form validation, and Resend for transactional email notifications. The user interface is crafted with Shadcn/UI components and Tailwind CSS, delivering a responsive and aesthetically modern experience.

Key features include OTP-based secure login, password reset via email, personalized public message pages, message inbox management (view, star, delete), sender blocking/allowing controls, profile customization with Cloudinary image uploads, real-time email alerts for new messages, and role-based access control with an admin analytics dashboard.

The project follows an Agile development methodology with iterative sprints, employing the App Router architecture of Next.js 15 for server-side rendering and API route handling. The system architecture adopts a Model-View-Controller (MVC) pattern with clear separation of concerns across models, API routes, and React components.

This report documents the complete development lifecycle of Secret Box, including its introduction, methodology, tools and technologies, implementation details, key outcomes, and future scope for enhancement.

1. Introduction

In the modern digital landscape, privacy has become a paramount concern. Social media platforms and messaging applications often require user identification, limiting the scope for truly anonymous feedback and communication. There exists a genuine need for a platform where individuals can share honest thoughts, confessions, or feedback without the fear of being identified.

1.1 Problem Statement

Most existing communication platforms mandate user registration for both senders and receivers, creating barriers to anonymous interaction. Users who wish to receive candid feedback—whether students, professionals, or content creators—lack a simple, privacy-respecting tool that allows anyone to send messages without creating an account.

1.2 Proposed Solution

Secret Box addresses this gap by providing a privacy-first anonymous messaging platform. Each registered user receives a unique, shareable link. Anyone with this link can send anonymous messages without signing up or logging in. The platform gives receivers complete control over their inbox, including the ability to view, star, delete messages, and toggle message acceptance on or off.

1.3 Objectives

• Develop a full-stack web application enabling anonymous message submission via unique user links.
• Implement secure authentication with OTP verification, social login (Google & GitHub), and password reset functionality.
• Provide users with a comprehensive dashboard for message management, profile customization, and privacy controls.
• Ensure a responsive, modern UI/UX using Shadcn/UI and Tailwind CSS with dark mode support.
• Implement role-based access control with an admin analytics dashboard.
• Deploy email notification services for OTP delivery and new message alerts.

1.4 Scope

The scope of Secret Box encompasses user registration and authentication, anonymous message sending and receiving, inbox management, profile customization, sender control mechanisms, email notifications, and administrative analytics. The application is designed as a web-based solution accessible across all modern browsers and devices.

2. Methodology

The development of Secret Box followed the Agile Software Development methodology, employing iterative development cycles (sprints) to incrementally build and refine features. This approach allowed for continuous feedback integration and adaptive planning throughout the project lifecycle.

2.1 Development Approach

The project adopted a component-driven development approach using React and Next.js. The application architecture follows the Model-View-Controller (MVC) pattern:

• Model Layer: MongoDB schemas defined using Mongoose (User and Message models) with TypeScript interfaces for type safety.
• Controller Layer: Next.js API routes (App Router) serving as RESTful endpoints for all backend operations.
• View Layer: React components organized into logical groups—authentication, dashboard, home, send-message, and common UI components.

2.2 Development Phases

Phase 1 – Planning & Design: Requirements gathering, database schema design, UI/UX wireframing, and technology stack selection.
Phase 2 – Core Development: Implementation of authentication system (NextAuth.js with credentials, Google, and GitHub providers), database models, and core API routes.

Phase 3 – Feature Development: Anonymous messaging flow, dashboard components (overview, messages, profile, settings), and email notification integration using Resend.

Phase 4 – Advanced Features: Role-based access control, admin analytics dashboard, sender blocking/allowing, Cloudinary image upload integration, and OTP verification system.

Phase 5 – Testing & Deployment: End-to-end testing, bug fixes, performance optimization, and production deployment.

2.3 System Architecture

The system follows a three-tier architecture: the Presentation Tier (React components with Shadcn/UI), the Application Tier (Next.js API routes with middleware for authentication and route protection), and the Data Tier (MongoDB Atlas cloud database). The middleware layer handles JWT-based session validation and role-based route protection.

3. Tools and Technologies

3.1 Frontend Technologies

• Next.js 15: A React-based meta-framework providing server-side rendering (SSR), static site generation (SSG), and the App Router architecture for file-system-based routing.

• TypeScript: A statically typed superset of JavaScript that enhances code quality, developer productivity, and maintainability through compile-time type checking.

• React 19: The latest version of the React library for building component-based user interfaces with hooks and functional components.

• Tailwind CSS 4: A utility-first CSS framework for rapidly building custom, responsive designs without writing traditional CSS.

• Shadcn/UI: A collection of beautifully designed, accessible, and customizable React components built on Radix UI primitives.

• Framer Motion: A production-ready animation library for React that powers smooth transitions and micro-interactions.

3.2 Backend Technologies

• Next.js API Routes: Server-side API endpoints built into the Next.js framework, eliminating the need for a separate backend server.

• NextAuth.js v4: A complete authentication solution for Next.js supporting credentials-based login, OAuth providers (Google, GitHub), JWT sessions, and callback customization.

• MongoDB with Mongoose: A NoSQL document database paired with an elegant ODM (Object Data Modeling) library for schema definition, validation, and query building.

• Zod: A TypeScript-first schema validation library used for form validation (sign-up, sign-in, message submission, profile update schemas).

• Bcrypt.js: A library for hashing and salting passwords to ensure secure credential storage.

3.3 External Services

• Resend: A modern email API service used for sending OTP verification codes and new message notification emails via React Email templates.

• Cloudinary: A cloud-based image management platform used for user profile image uploads, storage, and delivery.

• MongoDB Atlas: A fully managed cloud database service hosting the application's MongoDB instance.

3.4 Development Tools

• VS Code: Primary code editor with TypeScript and ESLint integration.

• Git & GitHub: Version control and collaborative development platform.

• npm: Package manager for dependency management.

• ESLint: Static code analysis tool for identifying and fixing code quality issues.

4. Implementation

4.1 Database Design

The application uses two primary MongoDB collections:

User Model: Stores user information including name, username (unique), email (unique), hashed password, verification code and expiry, account verification status, message acceptance toggle, reset token for password recovery, OTP session data, profile image URL, headline, custom question, social login IDs (Google/GitHub), role (admin/user), message references, and creation timestamp.

Message Model: Stores individual messages with content, creation timestamp, read status, and starred status. Messages are referenced from the User model via ObjectId references.

4.2 Authentication System

The authentication system is built on NextAuth.js with three providers:

• Credentials Provider: Email/password-based authentication with bcrypt password verification and OTP-based email verification during signup.

• Google Provider: OAuth 2.0-based social login that automatically creates or links user accounts.

• GitHub Provider: OAuth-based social login for developer-friendly account creation.

The NextAuth configuration includes custom callbacks for JWT token enrichment (adding user ID, username, verification status, and role) and session management. The middleware (middleware.ts) protects dashboard routes, redirects authenticated users away from auth pages, and enforces admin-only access to the analytics dashboard.

4.3 API Routes

The application exposes 17 RESTful API endpoints through Next.js API routes:

• /api/auth/[...nextauth] – Authentication handler (sign in, sign out, session management)

• /api/send-message – Accepts anonymous messages for a specified username

• /api/get-messages – Retrieves all messages for the authenticated user

• /api/message-delete – Deletes a specific message by ID

• /api/toggle-star – Toggles the starred status of a message

• /api/mark-as-read – Marks a message as read

• /api/accept-messages – Toggles the user's message acceptance status

• /api/check-username – Validates username availability during registration

• /api/verify-otp – Verifies the OTP code during email verification

• /api/resend-otp – Resends the OTP verification code

• /api/reset-link – Sends a password reset link via email

• /api/reset-password – Processes password reset with token validation

• /api/get-user – Retrieves user profile data

• /api/update-user – Updates user profile information

• /api/upload – Handles profile image upload to Cloudinary

• /api/delete-account – Permanently deletes a user account and all associated data

• /api/admin – Provides analytics data for admin dashboard

4.4 Frontend Components

The frontend is organized into modular component groups:

• Authentication Components: SignIn, SignUp, VerifyOtp, ResetLink, ResetPassword – handling the complete auth flow with form validation via Zod and React Hook Form.

• Dashboard Components: Overview (user statistics and activity summary), MessagesTable (inbox with search, filter, star, delete), ProfileForm (profile editing with image upload), ShareLink (unique link sharing), SenderControl (message acceptance toggle), DeleteAccount (account removal), and Analytics (admin-only user management).

• Home Components: HeroSection (landing page hero with animations), Features (feature showcase), CtaSection (call-to-action), MarqueeDemo (scrolling testimonials).

• Send Message Components: UserProfile (public profile display) and MessageForm (anonymous message submission form).

• Common Components: Navbar (responsive navigation with auth-aware menu), Logo, UserMenu, NotificationMenu, InfoMenu.

5. Major Findings / Outcomes / Results

5.1 Key Outcomes

The Secret Box platform was successfully developed and deployed as a fully functional anonymous messaging application. The key outcomes include:

• Complete Authentication System: Multi-provider authentication (credentials + Google + GitHub) with OTP verification, password reset, and JWT-based session management was successfully implemented.

• Anonymous Messaging: The core feature allowing anyone to send messages via a unique user link without registration was implemented with proper validation and rate limiting.

• Responsive Dashboard: A comprehensive user dashboard with message management (view, star, delete, search, filter), profile customization, privacy controls, and settings was delivered.

• Admin Analytics: Role-based access control with an admin dashboard providing user statistics and management capabilities was implemented.

• Email Integration: Automated email notifications for OTP verification and new message alerts were integrated using Resend with custom React Email templates.

5.2 Technical Achievements

• Server-Side Rendering: Leveraged Next.js 15 App Router for optimal performance with server-side rendering and streaming.

• Type Safety: Full TypeScript implementation across frontend and backend, ensuring compile-time error detection and improved code maintainability.

• Database Optimization: Implemented connection pooling and singleton pattern to prevent database connection overload during Next.js hot reloads in development mode.

• Security: Bcrypt password hashing, JWT token management, middleware-based route protection, and OTP-based verification ensure robust security.

• Modern UI/UX: Dark mode support, smooth animations via Framer Motion, responsive design, and accessible Shadcn/UI components deliver a premium user experience.

5.3 Challenges and Solutions

• Challenge: Preventing multiple database connections during Next.js development hot reloads. Solution: Implemented a global singleton pattern for Mongoose connection management.

• Challenge: Handling JSON parsing errors from API responses. Solution: Added robust response validation (checking res.ok before parsing) across all client-side API calls.

• Challenge: Managing complex authentication flows with multiple providers. Solution: Utilized NextAuth.js callbacks for JWT enrichment and custom session handling.

6. Conclusion and Future Scope

6.1 Conclusion

The Secret Box project has been successfully developed as a comprehensive, privacy-first anonymous messaging platform that fulfills all defined objectives. The application demonstrates the effective use of modern web development technologies—Next.js 15, TypeScript, NextAuth.js, MongoDB, Tailwind CSS, and Shadcn/UI—to build a secure, scalable, and user-friendly full-stack application.

The project provided valuable hands-on experience in full-stack development, covering database design, RESTful API development, authentication systems, responsive UI design, email integration, cloud image management, and deployment. The Agile methodology enabled iterative refinement, resulting in a polished and feature-rich final product.

6.2 Future Scope

The following enhancements are planned for future iterations:

• AI-Powered Message Suggestions: Integration of OpenAI API (already included in dependencies) to generate suggested messages for senders, improving engagement.

• Real-Time Notifications: Implementation of WebSocket-based real-time message notifications instead of email-only alerts.

• Message Analytics: Detailed analytics for users showing message trends, peak activity times, and engagement metrics.

• Mobile Application: Development of native mobile applications using React Native for iOS and Android platforms.

• End-to-End Encryption: Implementation of message encryption to further enhance privacy and security.

• Redis Caching: Full integration of Redis for session management, rate limiting, and caching to improve application performance.

• Multi-Language Support: Internationalization (i18n) to support multiple languages and expand the user base globally.

References

1. Next.js Documentation. (2026). https://nextjs.org/docs

2. NextAuth.js Documentation. (2026). https://next-auth.js.org/

3. MongoDB Documentation. (2026). https://www.mongodb.com/docs/

4. Mongoose Documentation. (2026). https://mongoosejs.com/docs/

5. Tailwind CSS Documentation. (2026). https://tailwindcss.com/docs

6. Shadcn/UI Documentation. (2026). https://ui.shadcn.com/

7. TypeScript Documentation. (2026). https://www.typescriptlang.org/docs/

8. Zod Documentation. (2026). https://zod.dev/

9. Resend Documentation. (2026). https://resend.com/docs

10. Cloudinary Documentation. (2026). https://cloudinary.com/documentation

11. Framer Motion Documentation. (2026). https://www.framer.com/motion/

12. React Hook Form Documentation. (2026). https://react-hook-form.com/